

Day 2 실습 — 사내 문서 Q&A 만들기

- 인제스트부터 근거 있는 답까지
- 검색을 먼저 눈으로 본다
- 골든 세트로 측정하고 고친다
- 100분 · 2인 1조 권장

오늘 만들 것 — 문서 Q&A

- 어제 만든 API 위에 근거를 붙인다
- 6단계 파이프라인을 직접 만든다
- 막히면 ch07_rag 예제를 연다

구분	무엇을	확인 방법
목표	사내 규정으로 답하고 출처를 붙이는 API	POST /lab2/ask
파이프라인	읽기 → 분할 → 임베딩 → 저장 → 검색 → 생성	6단계를 코드로
응답	답변 + 출처 + 근거 사용 여부	구조화 출력(record)
측정	골든 세트 10문항	통과율과 실패 유형 분류
실험	청크 크기 · top-k 조합	실험표 A~D를 채운다
산출물	lab2 패키지 · golden.json	완료 기준 8개
시간	100분	인제스트 25 · 답변 35 · 측정 25 · 정리 15

준비 — 문서 세 개와 골든 세트

- 문서는 짧고 서로 다른 세 개면 충분하다
- 골든 세트에 답이 없어야 하는 질문을 꼭 넣는다
- 정답 기준을 먼저 정하고 시작한다

```
src/main/resources/lab2-docs/
```

```
|─ return-policy.md      반품·교환 (단순변심 7일 · 배송비 고객 부담)
|─ shipping-policy.md    배송 정책 (제주·도서 추가비 · 평균 2~3일)
└─ membership.md        등급·포인트 (실버 1% · 골드 3%)
```

```
# golden.json – 정답이 정해진 질문 10개(아래는 4개만)
```

```
[
  {"q": "단순 변심 반품은 며칠 이내인가요?", "must": ["7일"], "src": "return-policy"},
  {"q": "제주도는 배송비가 더 드나요?", "must": ["추가"], "src": "shipping-policy"},
  {"q": "골드 등급 적립률은?", "must": ["3%"], "src": "membership"},
  {"q": "우주 배송도 되나요?", "must": ["확인되지"], "src": null}
]
```

```
# ↑ 마지막 문항이 핵심이다 – 문서에 없는 것을 지어내지 않는지 본다
```

```
# 표현을 바꾼 질문도 두어 개 넣는다(검색이 표현에 얼마나 흔들리는지 본다)
```

```
# "물건 돌려보내려면 며칠 안에 해야 해요?" → 같은 정답이어야 한다
```

Step 1 — 인제스트: 메타데이터가 절반

- 읽기 → 분할 → 메타데이터 → 저장
- 출처·버전은 이 시점에만 넣을 수 있다
- 재인제스트로 중복이 쌓이지 않게 만든다

```
@Service
public class Lab2IngestService {
    private final VectorStore vectorStore;

    public IngestResult ingest(Resource doc, String source, String version) {
        var reader = new TextReader(doc);           // .md → Document
        reader.getCustomMetadata().put("source", source); // 나중에 못 넣는다
        reader.getCustomMetadata().put("version", version);

        var splitter = TokenTextSplitter.builder()
            .withChunkSize(400)           // 토큰 기준 - 문서 성격에 맞춘다
            .withMinChunkSizeChars(200)
            .build();
        List<Document> chunks = splitter.apply(reader.get());

        vectorStore.delete(new FilterExpressionBuilder() // 재색인 -
            .eq("source", source).build());           //   같은 출처를 지우고
        vectorStore.add(chunks);                       //   다시 넣는다
        return new IngestResult(source, chunks.size());
    }
}

// 확인: POST /lab2/ingest → [{"source":"return-policy","chunks":7}, ...]
```

Step 2 — 검색을 먼저 눈으로 본다

- 답변보다 검색 API를 먼저 만든다
- 점수를 반드시 노출한다 — 감으로 판단하지 않는다
- 세 질문의 결과를 나란히 비교한다

```

@GetMapping("/lab2/retrieve")          // 답변을 만들기 전에 이것부터 만든다
public List<Chunk> retrieve(@RequestParam String q,
                           @RequestParam(defaultValue = "4") int topK) {
    return vectorStore.similaritySearch(SearchRequest.builder()
        .query(q).topK(topK)
        .similarityThreshold(0.5)      // 낮은 점수는 근거가 아니다
        .build())
        .stream()
        .map(d -> new Chunk(d.getMetadata().get("source").toString(),
                           d.getScore(),                          // 점수를 노출한다
                           snippet(d.getText(), 120)))
        .toList();
}

// 실습 - 아래 세 질문의 결과를 나란히 놓고 비교한다
// ① "반품 기한"                기대: return-policy 상위
// ② "물건 돌려보내려면 며칠 안에?"    기대: 같은 문서 - 표현이 달라도 찾는가
// ③ "우주 배송"                기대: 점수가 전부 낮다 → 근거 없음

```

Step 3 — 근거로 답하기

- 근거가 없으면 모델을 부르지 않는다
- 프롬프트에 거절 지시를 반드시 넣는다
- 응답은 문자열이 아니라 객체로 받는다

```
public AnswerDto ask(String question) {
    var docs = retrieve(question, 4);
    if (docs.isEmpty()) {
        return AnswerDto.unknown();          // 근거가 없으면 모델을 부르지 않는다
    }
    return chatClient.prompt()
        .system("""
        아래 [근거]만 사용해 답한다. 근거에 없으면 "확인되지 않습니다"라고 답한다.
        추측하지 않는다. 답변 끝에 사용한 출처를 [출처: 파일명] 형식으로 남긴다.
        """)
        .user(u -> u.text("[근거]\n{context}\n\n[질문] {question}")
            .param("context", format(docs))
            .param("question", question))
        .call()
        .entity(AnswerDto.class);           // 구조화 출력 - 문자열 파싱 금지(6장)
}

record AnswerDto(String answer, List<String> sources, boolean grounded) {
    static AnswerDto unknown() { return new AnswerDto("확인되지 않습니다.", List.of(), false); }
}
```

Step 4 — 골든 세트로 측정

- 느낌으로 고치면 고쳤는지 알 수 없다
- 기준선을 코드에 박아 둔다
- 실패한 질문의 답을 반드시 읽는다

```
@Test // 모델을 부르므로 기본 테스트에서는 제외한다 (./gradlew test -Peval)
void 골든_세트_평가() throws Exception {
    var golden = mapper.readValue(resource("golden.json"),
                                   new TypeReference<List<Golden>>() {});

    int pass = 0;
    for (Golden g : golden) {
        AnswerDto a = service.ask(g.q());
        boolean hit = g.must().stream().allMatch(k -> a.answer().contains(k));
        boolean cite = g.src() == null
            || a.sources().stream().anyMatch(s -> s.contains(g.src()));
        if (hit && cite) { pass++; }
        else { log.warn("실패: {}\n 답변: {}\n 출처: {}", g.q(), a.answer(), a.sources()); }
    }
    log.info("통과 {}/{}", pass, golden.size());
    assertThat(pass).isGreaterThanOrEqualTo(8); // 기준선을 코드에 박아 둔다
}

// 실패를 두 종류로 나눠 적는다 - 고칠 곳이 완전히 다르다
// ㉠ 근거를 못 찾았다 → 청킹 · 임베딩 · top-k · 질문 변환
// ㉡ 찾기도 잘못 답했다 → 프롬프트 · 모델 · 근거 포맷
```

Step 5 — 실험표를 채운다

- 정답은 문서마다 다르다 — 감이 아니라 기록으로 정한다
- 한 번에 하나만 바꾼다
- 채운 팀과 안 채운 팀의 차이가 실서비스 품질이 된다

조합	청크	top-k	통과율 / 관찰한 것
A (기준)	400토큰 · 겹침 0	4	___ / 10 ← 기준선
B (작게)	200토큰	4	___ / 10 정확하지만 맥락이 부족한가?
C (크게)	800토큰	4	___ / 10 맥락은 넓지만 잡음이 늘었나?
D (넓게)	400토큰	8	___ / 10 근거도 비용도 함께 늘었나?
E (엄격)	400토큰 · threshold 0.7	4	___ / 10 거절이 늘었나? 정답도 거절했나?
F (겹침)	400토큰 · 겹침 20%	4	___ / 10 잘린 문장 문제가 줄었나?

자주 막히는 지점 — Day 2 실습

- 열에 아홉은 이 표 안에 있다
- 진단 순서는 언제나 같다
- 검색 → 프롬프트 → 그다음 모델

증상	원인	해결
검색 결과가 늘 비어 있다	임베딩 모델 미설정 · 미다운로드	키 확인 또는 임베딩 모델 설정 확인(text-embedding-3-small)
인제스트할수록 답이 나 빠진다	중복 적재	재색인(삭제 후 삽입) 또는 id 고정
근거는 맞는데 답이 틀리다	프롬프트에 거절·인용 지시 없음	시스템 프롬프트 보강
항상 "확인되지 않습니다"	임계값이 너무 높다	threshold를 낮추고 점수 분포부터 확인
출처가 안 붙는다	메타데이터를 안 넣었다	인제스트 시점에 source 주입
컨텍스트 초과 오류	top-k · 청크가 너무 크다	k 축소 또는 근거를 요약해 투입
응답이 5초를 넘는다	k 과다 · 임베딩 호출 반복	구간별 시간 측정부터